

0. 考试导航 Exam Map

| 优先 | 模块                      | 典型题型 / 动作                              |
|----|-------------------------|----------------------------------------|
| S  | Search (Uninf./Inf.)    | 绘图, 写 visited order + path             |
| S  | CSP                     | Variables/Domains/Constraints + graph  |
| S  | Minimax + $\alpha\beta$ | 自底向上算 value , 标剪枝                      |
| A  | Expectimax              | 每动作算 EU , 选最大                          |
| A  | Linear Prog.            | 文字题→LP ; 图解找 vertex                    |
| B  | Optim./Convex           | MCQ : feasible / convex / local=global |

1. Search 基本定义

Search problem: state space 状态集; successor function 动作+代价; start state ; goal test ; solution 动作序列; path cost 总代价。

State-space graph vs Search tree: graph 每 state 只 1 次; tree 每 node 是路径, 同 state 可多次。Fringe=已生成未展开; Expand=取出+生成后继。

2. 通用做题模板 ( visited+path )

1. 画表 Visited|Fringe|Parent|g|h|f. 2. 每步选要 visit 的 node : DFS=最深; BFS=最浅; UCS=最小生成树 ( 不是 generated 时 )。3. Goal popped 才停 ( 不是 generated 时 )。4. Tie-break 按题 ( 常 alphabetical )。5. 写 : visited order ( 含 S, G ) ; solution path ( 沿 parent 回溯 ) ; path cost。

3. DFS / BFS / IDDFS

DFS=stack , 深; BFS=queue , 浅, unit cost 时 optimal。

IDDFS=不断增 limit 跑 DFS:

limit=0: DFS from S, depth<=0  
limit=1: restart, depth<=1  
limit=2: restart, depth<=2  
... stop when G visited

每轮重启 : visited 重复出现 S ; solution path 写最后那条。  
IDDFS combines BFS-like completeness with DFS-like memory.

4. UCS / Greedy / A\*

UCS: fringe 按  $g(n)$  升序 pop. goal popped 才停 ; 同 state 更低 cost 替换 ; 非负边权 ⇒ optimal。

Greedy: fringe 按  $h(n)$  升序。快但不 optimal , 可能被骗。

A\*:  $f(n) = g(n) + h(n)$ 。每次 pop min  $f$  ; goal popped 停 ; 回溯 parent 得 path。Admissible:  $0 \leq h(n) \leq h^*(n)$  ( 不高估 )。Tree search + admissible ⇒ optimal。Consistent:  $h(n) \leq c(n, n') + h(n')$  ; graph search optimal。

A\* expands nodes in increasing  $f = g + h$ . With admissible h, tree search is optimal.

5. Search 性质速查

| Algo   | Priority   | Comp. | Optimal     | 常见坑            |
|--------|------------|-------|-------------|----------------|
| DFS    | deepest    | 否     | 否           | 找左边的, 不管 cost  |
| BFS    | shallowest | 是     | 仅 unit cost | 内存大            |
| UCS    | min $g$    | 是     | 是           | goal popped 才停 |
| Greedy | min $h$    | 否     | 否           | 只看 h 易被骗       |
| A*     | min $g+h$  | 是     | adm/cons.是  | f 每步更新         |
| IDDFS  | DFS+limit  | 是     | 仅 unit cost | visited 重复     |

6. CSP 约束满足问题

CSP=Variables  $X_1 \dots X_n$  + Domains  $D_i$  + Constraints ( 合法值组合 )。state=部分赋值; goal=完整赋值满足全约束; successor=给一未赋变量赋值。

Formulation 三件套 ( 万能模板 )

Variables:  $X_1=..., X_2=..., \dots$   
Domains:  $D(X_1)=\{...\}$ ,  $D(X_2)=\{...\}$   
Constraints:  
for incompatible  $(X_i, X_j)$ :  $X_i \neq X_j$   
explicitly:  $X_1!=X_2$ ,  $X_2!=X_3$ , ...

One variable per object, domain = legal values, constraints exclude incompatible assignments.

排课 Course Scheduling 模板

1. Variables: 每门课 1 个  $X_i$ =professor of class  $i$ 。2. Domains:  $D(X_i)$ =能教此课的 professor。3. Constraints: 时间重叠两课  $X_i \neq X_j$ 。4. Constraint graph: node=class; edge=两课时间重叠。5. 写一个解:  $X_1=C$ ,  $X_2=B$ ,  $X_3=C$ ,  $X_4=A$ ,  $X_5=B$ 。

7. Backtracking + 启发式

Backtracking: DFS+一次赋 1 个变量+边赋值查 constraint, 违反则回溯。赋值顺序 commutative ; 可做 incremental goal test。

MRV (Minimum Remaining Values): 选剩余合法值最少的变量 ( fail fast )。MRV picks the most constrained variable.

LCV (Least Constraining Value): 选对邻居影响最小的值 ( 给别人留活路 )。LCV picks the value ruling out the fewest options for neighbors.

Forward Checking: 赋  $A_i$  后, 删每个未赋邻居 domain 中冲突值; 任一空 ⇒ failure。

Arc Consistency (AC-3):  $X \rightarrow Y$  consistent iff  $\forall x \in D(X), \exists y \in D(Y)$  满足约束。delete from tail ;  $X$  失值 ⇒ 重查邻居 ; 比 FC 更早发现失败 ; 单独不能解整 CSP。

8. Minimax 极小化极大

两人零和确定博弈。MAX 取 max ; MIN 取 min ; leaf=utility。

做题: 1.从 leaf 开始 ; 2.MIN=min(children) ; 3.MAX=max(children) ; 4.层层往上填 ; 5.root 选最大 value 的 action。

Minimax assumes optimal opponent and chooses the move maximizing utility under worst-case responses.

9. Alpha-Beta Pruning

$\alpha$ =MAX 已能保证最好 ;  $\beta$ =MIN 已能保证最好。不改变 minimax value。

At MAX:  $v=\max(v, \text{child})$ ;  $a=\max(a, v)$   
prune if  $a \geq b$   
At MIN:  $v=\min(v, \text{child})$ ;  $b=\min(b, v)$   
prune if  $a \geq b$

考场步骤: 按题目 exploration order 左→右 DFS ; 每 node 写当前  $\alpha, \beta$  ; 算完 child 更新 ;  $\alpha \geq \beta$  即剪后续 sibling ; 图上 slash pruned leaves。

常见坑: 必按题序遍历 ; pruned leaf 不算 explored ; 顺序好剪枝多 ; 剪枝结果与 minimax 一致。

解释句: This branch is pruned because the ancestor already has a better guaranteed option, so it cannot affect the root decision.

10. Expectimax 随机博弈

Minimax vs Expectimax: Minimax 对手最优取 min ; Expectimax 概率取加权平均。适用 dice / random ghost / stochastic action。

公式 : MAX:  $V(s) = \max_a V(\text{succ})$  ; Chance:  $V(s) = \sum_i P(o_i) V(o_i)$ 。

做题: 1.列我方 candidate moves ; 2.每 move 列 opponent response 与 P ; 3.每 outcome 写 utility (win=+1, draw=0, lose=-1) ; 4.算 EU  $= \sum P \cdot U$  ; 5.选 EU 最大者。

Relative weight→P: "middle 是 top 的 3 倍, bottom 是 middle 的 3 倍" ⇒ 权重 1:3:9 ; 两选一 1:3 ⇒ 0.25, 0.75。  $P(i) = w_i / \sum w_j$ 。

EU(A) = 0.25\*(-1) + 0.75\*(1) = 0.5  
EU(B) = 0.25\*(1) + 0.75\*(0) = 0.25  
=> Choose A (0.5 > 0.25)

At chance nodes, take a probability-weighted average of child values.

11. Linear Programming 标准式

最大化  $z = c_1x_1 + c_2x_2 + \dots + c_nx_n$   
s.t.  $a_{11}x_1 + a_{12}x_2 \leq b_1$   
 $a_{21}x_1 + a_{22}x_2 \leq b_2$   
 $x \geq 0, y \geq 0$   
( $x, y$  in  $Z$  if integer)

文字题→LP 三步模板

1. Variables: Let  $x$ =amount of A; Let  $y$ =amount of B. 2. Constraints: 每条资源限制译成线性不等式 + 非负 + 整数 ( 如需 )。3. Objective: profit→max; cost→min。

Define decision variables, translate each resource limit into a linear constraint, optimize the linear objective.

典型例: Printed Circuits

$x$ =#TypeA,  $y$ =#TypeB  
max  $P = 5x + 12y$   
s.t.  $20x + 10y \leq 200$   
 $10x + 20y \leq 120$   
 $10x + 30y \leq 150$   
 $x, y \geq 0, x, y$  in  $Z$

12. Graphical Solution 图解法

仅 2 变量。1.每不等式画边界 ; 2.判断保留侧 ( 代原点试 ) ; 3.交集得 feasible region ; 4.找 vertices ( 约束相交点 ) ; 5.每 vertex 代入 obj, max/min 取最值。

为什么只看顶点 : For an LP, an optimal solution occurs at a vertex of the feasible region (if it exists)。

Profit/Cost Line:  $C = ax+by$  同直线 profit 同。Max: 沿增方向平移, 最后碰到的可行点=最优; Min: 反向, 最先碰到=最优。

13. Integer Programming (IP)

LP+整数约束 :  $x \in \mathbb{Z}^n$  ; 二元  $\{0, 1\}^n$  ; MILP=部分整数。

LP Relaxation: 丢整数约束→relaxed LP。Max: relaxed  $\geq$  IP ; Min: relaxed  $\leq$  IP。LP 解不能随便 round ( 可能 infeasible 或非最优 )。

Branch and Bound 模板

1. 解 LP relaxation ; 若已整数→candidate incumbent。2. 若  $x = 2.3$  分数 : 分支  $x \leq 2$  与  $x \geq 3$ 。3. 每分支再解 LP relaxation。4. Prune 若 : infeasible / bound 比 incumbent 差 / 已整数。5. 最优 incumbent 即 IP 最优。

14. Optimization / Convexity

三要素: min / max  $f(x)$  s.t.  $x \in F$  ;  $x$ =变量 ;  $F$ =feasible set ;  $f$ =objective。min  $f \Leftrightarrow$  max  $-f$ 。

Convex Set:  $\forall x, y \in F, \theta \in [0, 1] : \theta x + (1 - \theta)y \in F$ 。 $\mathbb{R}^n, \emptyset, \{x_0\}$  凸 ; 凸集交凸 ; 并不一定凸 ;  $\mathbb{Z}^n$  非凸。

Convex Function:  $f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$ 。图像在 chord 下方。一元  $f'' \geq 0 \Rightarrow$  凸 ; 多元 Hessian  $\succeq 0 \Rightarrow$  凸。非负加权和、线性变换保凸。

Convex Optimization 关键: local optimum = global optimum。In a convex optimization problem, every local optimum is also a global optimum。

15. 易错点 Top-10

1. UCS / A\* goal popped 才停。2. Greedy 不 optimal : 只看  $h$  易被骗。3. IDDFS visited 重复 : 每轮从 S 重启。4. Admissible:  $0 \leq h \leq h^*$  ; A\* tree search 用 admissible 即 optimal ; graph search 需 consistent。5. BFS 仅 unit cost 时 optimal。6. CSP backtracking 一次只赋 1 变量 ; commutative。7. MRV=最少剩余 ; LCV=影响最小 ( 别搞反! )。8. FC 比 AC 弱 : AC 还要回头检查邻居。9. Alpha-Beta 必按题序 ; 剪枝不改 minimax value。10. LP relaxation 不能随便 round ; 最优在 vertex。

16. 答题模板速查（背模板就能写）

Search visited order

Algorithm: UCS / A\* / IDDFS  
Tie-break: alphabetical  
Goal test: when popped/visited

Visited | Fringe (after exp) | g | h | f | Par |

|-----|-----|-----|-----|

| ... | ... | | | | |

Visited order: S, ..., G  
Solution path: S -> ... -> G  
Path cost: ...

CSP formulation

Variables: Xi = ...  
Domains: D(Xi) = {...}  
Constraints:  
for incompatible (Xi,Xj): Xi != Xj  
explicitly: ...

Constraint graph:  
Nodes = variables  
Edges = binary constraints

One valid solution: X1=..., X2=..., ...

Minimax / Alpha-Beta

Bottom-up:  
MAX = max(children)  
MIN = min(children)

Alpha-Beta:  
alpha = best MAX guarantee  
beta = best MIN guarantee  
prune when alpha >= beta

Pruned-branch sentence:  
"Cannot affect root decision because  
ancestor already has a better  
guaranteed value."

Expectimax

For each candidate move a:  
EU(a) = SUM P(outcome)\*U(outcome)  
Choose argmax\_a EU(a).

Probability from weights:  
P(i) = w\_i / SUM w\_j

LP formulation + solve

Let x = amount of A.  
Let y = amount of B.

max/min Z = ...x + ...y  
s.t. ...x + ...y <= ...  
...x + ...y <= ...  
x,y >= 0  
x,y in Z (if integer units)

Graphical:  
- draw feasible region  
- list vertices  
- eval objective at each  
- choose best vertex

17. 公式速记墙

Search:  $f(n) = g(n) + h(n), 0 \leq h \leq h^*$ 。

Minimax:  $V_{\max}(s) = \max_a V(s'), V_{\min}(s) = \min_a V(s')$ 。

Alpha-Beta: prune iff  $\alpha \geq \beta$ 。

Expectimax:  $V_{\text{ch}}(s) = \sum_i P(o_i)V(o_i)$ 。

P from weights:  $P(i) = w_i / \sum_j w_j$ 。

LP standard:  $\max c^\top x \text{ s.t. } Ax \leq b, x \geq 0$ 。

Convex set/fn:  $\theta x + (1 - \theta)y \in F, \theta \in [0, 1]; f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$ 。

Convex check:  $f'' \geq 0$  (1D) ; Hessian  $\succeq 0$  (多元)。

18. 答题英文金句（直接抄）

Search/A\*: A\* expands nodes in increasing order of  $f = g + h$ . With admissible h, A\* tree search is optimal because h never overestimates the true remaining cost.  
UCS: UCS is complete and optimal when step costs are non-negative since paths are expanded in increasing order of cost.  
IDDFS: IDDFS combines BFS-like completeness with DFS-like memory by re-running depth-limited DFS with growing limits.  
CSP: We formulate as a CSP with one variable per object, domains as legal values, and constraints excluding incompatible assignments.  
MRV/LCV: MRV picks the most constrained variable; LCV picks the value that rules out the fewest options for neighbors.  
Minimax: Minimax assumes the opponent plays optimally and chooses the move that maximizes utility under worst-case responses.  
Pruning: This branch cannot affect the root decision because an ancestor already has a better guaranteed value.  
Expectimax: Expectimax computes expected utility under probabilistic outcomes; at chance nodes it averages child values weighted by probability.  
LP: We define decision variables, translate each resource limit into a linear constraint, and optimize a linear objective. The optimum lies at a vertex of the feasible region.  
Convex: In a convex optimization problem, every local optimum is also a global optimum.

19. 主线一图（考前 1 分钟扫一遍）

|            |                                        |
|------------|----------------------------------------|
| Search     | 找路径：DFS/BFS/UCS/Greedy/A* + IDDFS      |
| CSP        | 给变量赋值满足约束：Backtracking + MRV/LCV/FC/AC |
| Minimax    | 决策博弈：MAX vs MIN + Alpha-Beta 剪枝        |
| Expectimax | 决策+概率：chance node = 加权平均               |
| LP         | 资源分配：变量+约束+目标，顶点最优                     |
| IP         | LP+整数：用 branch-and-bound，不能随便 round    |
| Convex     | 凸集+凸函数 $\Rightarrow$ local = global    |

20. 30 分钟最后冲刺

- A\*:  $f = g + h$  ; admissible 不高估 ; goal popped 才停。
- UCS: 按  $g$  最小 ; 非负边权  $\Rightarrow$  optimal。
- IDDFS: limit 0,1,2... 每轮重跑 DFS , visited 重复。
- CSP: V/D/C ; constraint graph 的 edge= binary constraint。
- MRV=最少 legal values; LCV=排除最少。
- FC=赋值后删邻居 domain ; AC=tail 每值在 head 有 support。
- Minimax: MAX=max , MIN=min。
- Alpha-Beta:  $\alpha \geq \beta$  剪枝 ; 不改 minimax value。
- Expectimax: chance node=加权平均。
- LP: 先定  $x, y$  ; 列 constraints ; 写 objective ; 最优在 vertex。
- IP: LP relaxation 不能随便 round ; 用 B&B。
- Convex: local opt = global opt。

21. 考场流程（按这个走）

1. 扫题 30 秒: 哪个模块 ? 哪种题型 ? 2. 翻这张 cheatsheet 对应模板。 3. 按模板填空写答案 , 不自己发挥。 4. Search: 写表+visited order+path+cost。 5. CSP: V/D/C+graph+1 个解。 6. Tree: 自底向上+标  $\alpha, \beta$ +slash pruned。 7. LP: 定义变量  $\rightarrow$  列约束  $\rightarrow$  写目标  $\rightarrow$  顶点法解。 8. 留 2 分钟对照模板检查。

22. 题型识别雷达（看到关键词就翻这页）

| 看到                                           | 翻到                 |
|----------------------------------------------|--------------------|
| graph + start/goal + heuristic               | S2 通用模板 + S4 A*    |
| visited order / path cost                    | S2 + S5 性质表        |
| "depth limit" / "iterative"                  | S3 IDDFS           |
| "course schedule" / "map color" / "N-queens" | S6 CSP + S7 启发式    |
| "professor"/"time slot"                      | S6 排课模板            |
| constraint graph                             | S6 第 4 步           |
| "adversary" / "game tree" / "two-player"     | S8 Minimax         |
| prune branches                               | S9 $\alpha, \beta$ |
| "random" / "dice" / "probability $p$ "       | S10 Expectimax     |
| "max profit" / "min cost" + 限制               | S11 LP + S12 图解    |
| "products integer" / "whole units"           | S13 IP / B&B       |
| "convex" / "feasible set" / "local-global"   | S14 Convex         |

23. CSP 排课题完整范例

题 : 5 门课 , 时间冲突如下 ; 3 位教授 {A,B,C} 各能教哪些。求 1 个赋值方案。

Variables: X1..X5 = professor of class i  
Domains:  
D(X1)={A,B,C}, D(X2)={A,C}  
D(X3)={B,C}, D(X4)={A,B}  
D(X5)={B,C}

Constraints (overlapping pairs):  
X1!=X2, X2!=X3, X2!=X4  
X3!=X4, X3!=X5, X4!=X5

Constraint graph:  
edges = {(1,2),(2,3),(2,4),(3,4),(3,5),(4,5)}

One valid solution:  
X1=C, X2=A, X3=B, X4=B (X3!=X4 fails!)  
retry: X3=C, X4=A, X5=B  
-> X1=C, X2=A, X3=C, X4=B, X5=C (...)  
final: X1=B, X2=A, X3=B, X4=A, X5=C OK

24. Expectimax 题完整范例

题 : 随机对手 row 概率 top:mid:bot=1:3:9。我方两 move A/B , response 概率与效用如下。

Move A: response top (P=1/4) -> -1  
response bot (P=3/4) -> +1  
EU(A) = 0.25\*(-1) + 0.75\*(+1) = +0.5

Move B: response top (P=1/4) -> +1  
response bot (P=3/4) -> 0  
EU(B) = 0.25\*(+1) + 0.75\*( 0) = +0.25

=> Choose A (EU larger)。

25. LP 文字题完整范例

题 : A/B 两产品 , 资源 1/2/3 限制如下 , 求最大利润。

Step1 vars: x = #A, y = #B  
Step2 cons: 20x +10y <= 200 [r1]  
10x +20y <= 120 [r2]  
10x +30y <= 150 [r3]  
x,y >= 0; x,y in Z

Step3 obj: max P = 5x + 12y  
Step4 graph: draw lines + axes; find  
feasible region; list vertices; eval  
P at each; pick max -> (x\*,y\*,P\*)

口诀: Search 看 fringe , CSP 看图 , Tree 看  $\alpha, \beta$  , LP 看顶点 , Convex 看 local=global。